9 API gateway and circuit breakers

This chapter covers

- Implementing edge services with Spring Cloud Gateway and Reactive Spring
- Configuring circuit breakers with Spring Cloud Circuit Breaker and Resilience4J
- Defining rate limiters with Spring Cloud Gateway and Redis
- Managing distributed sessions with Spring Session Data Redis
- Routing application traffic with Kubernetes Ingress

In the previous chapter, you learned several aspects of building resilient, scalable, and cost-effective applications using the reactive paradigm. In this chapter, the Spring reactive stack will be the foundation for implementing an API gateway for the Polar Bookshop system. An API gateway is a common pattern in distributed architectures, like microservices, used to decouple the internal APIs from the clients. When establishing such an entry point to your system, you can also use it to handle cross-cutting concerns, such as security, monitoring, and resilience.

This chapter will teach you how to use Spring Cloud Gateway to build an Edge Service application and implement an API gateway and some of those cross-cutting concerns. You'll improve the resilience of the system by configuring circuit breakers with Spring Cloud Circuit Breaker, defining rate limiters with Spring Data Redis Reactive, and using retries and timeouts just like you learned in the previous chapter.

Next, I'll discuss how to design stateless applications. Some states will need to be saved for the applications to be useful—you have already used relational databases. This chapter will teach you how to store web session state using Spring Session Data Redis, a NoSQL in-memory data store.

Finally, you'll see how to manage external access to the applications running in a Kubernetes cluster by relying on the Kubernetes Ingress API.

Figure 9.1 shows what the Polar Bookshop system will look like after completing this chapter.

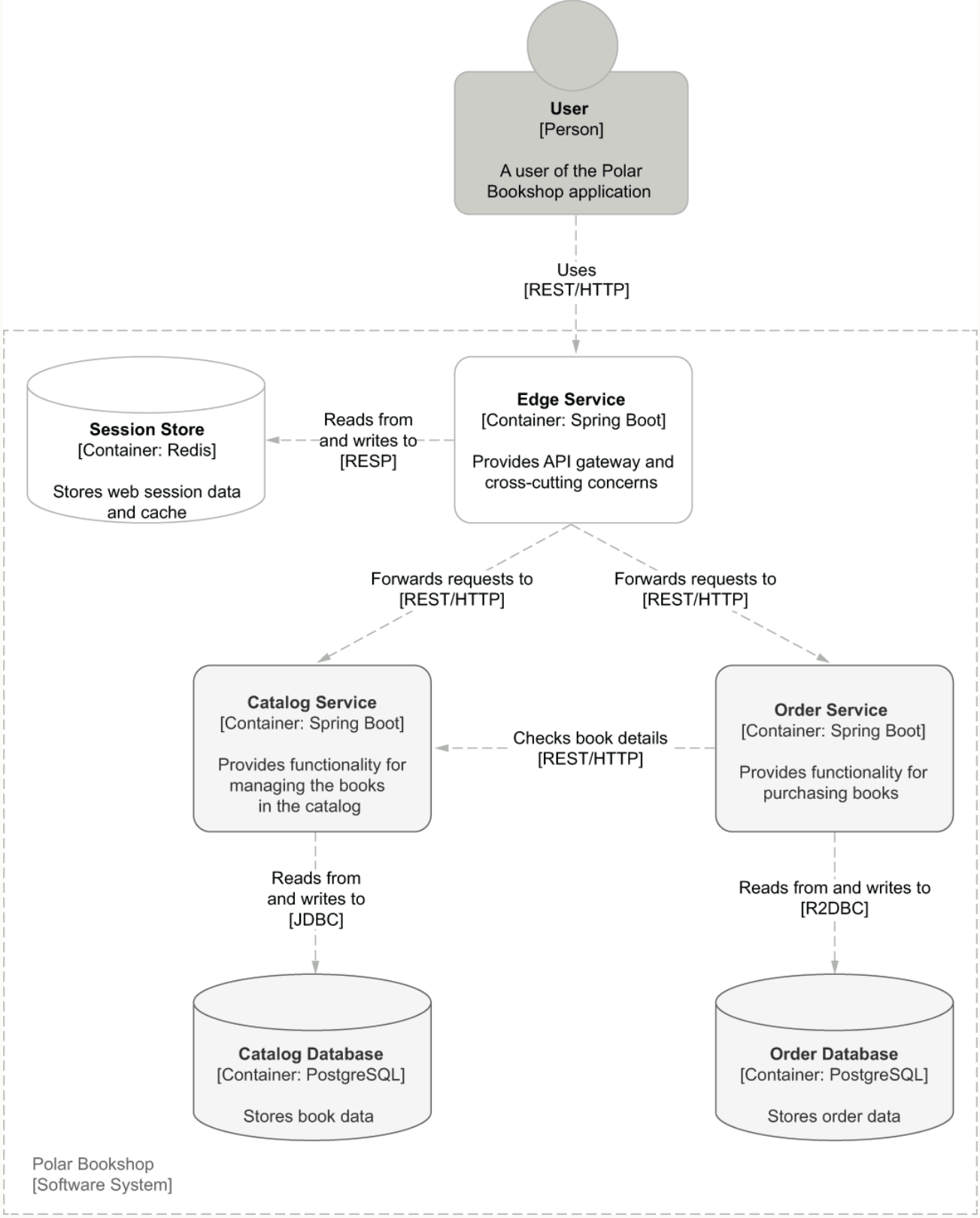


Figure 9.1 The architecture of the Polar Bookshop system after adding Edge Service and Redis

NOTE The source code for the examples in this chapter is available in the Chapter09/09-begin and Chapter09/09-end folders, containing the initial and final states of the project (**https://github.com/ThomasVitale/cloud-native-spring-in-action**).

9.1 Edge servers and Spring Cloud Gateway

Spring Cloud Gateway is a project built on top of Spring WebFlux and Project Reactor to provide an API gateway and a central place to handle cross-cutting concerns like security, resilience, and monitoring. It's built for developers, and it's a good fit in Spring architectures and heterogeneous environments.

An API gateway provides an entry point to your system. In distributed systems like microservices, that's a convenient way to decouple the clients from any changes to the internal services' APIs. You're free to change how your system is decomposed into services and their APIs, relying on the fact that the gateway can translate from a more stable, client-friendly, public API to the internal one.

Suppose you're in the process of moving from a monolith to microservices. In that case, an API gateway can be used as a *monolith strangler* and can wrap your legacy applications until they are migrated to the new architecture, keeping the process transparent to clients. In case of different client types (single-page applications, mobile applications, desktop applications, IoT devices), an API gateway gives you the option to provide a better-crafted API to each of them depending on their needs (also called the *backend-for-frontend* pattern). Sometimes a gateway can also implement the *API composition* pattern, letting you query and join data from different services before returning the result to a client (for example, using the new Spring for GraphQL project).

Calls are forwarded to downstream services from the gateway according to specified routing rules, similar to a reverse proxy. This way the client doesn't need to keep track of the different services involved in a transaction, simplifying the client's logic and reducing the number of calls it has to make.

Since the API gateway is the entry point to your system, it can also be an excellent place to handle cross-cutting concerns like security, monitoring, and resilience. *Edge servers* are applications at the edge of a system that implement aspects like API gateways and cross-cutting concerns. You can configure circuit breakers to prevent cascading failures when invoking the services downstream. You can define retries and timeouts for all the calls to internal services. You can control the ingress traffic and enforce quota policies to limit the use of your system depending on some criteria (such as the membership level of your users: basic, premium, pro). You can also implement authentication and authorization at the edge and pass tokens to downstream services (as you'll see in chapters 11 and 12).

However, it's important to remember that an edge server adds complexity to the system. It's another component to build, deploy, and manage in production. It also adds a new network hop to the system, so the response time will increase. That's usually an insignificant cost, but you should keep it in mind. Since the edge server is the entry point to the system, it's at risk of becoming a single point of failure. As a basic mitigation strategy, you should deploy at least two replicas of an edge server following the same approach we discussed for configuration servers in chapter 4.

Spring Cloud Gateway greatly simplifies building edge services, focusing on simplicity and productivity. Furthermore, since it's based on a reactive stack, it can scale efficiently to handle the high workload naturally happening at the edge of a system.

The following section will teach you how to set up an edge server with Spring Cloud Gateway. You'll learn about routes, predicates, and filters, which are the building blocks of the gateway. And you'll apply the retry and timeout patterns you learned in the previous chapter to the interactions between the gateway and the downstream services.

NOTE If you haven't followed along with the examples implemented in the previous chapters, you can refer to the repository accompanying the book and use the project in Chapter09/09-begin as a starting point (**https://github.com/ThomasVitale/cloud-native-spring-in-action**).

9.1.1 Bootstrapping an edge server with Spring Cloud Gateway

The Polar Bookshop system needs an edge server to route traffic to the internal APIs and address several cross-cutting concerns. You can initialize our new Edge Service project from Spring Initializr (**https://start.spring.io**), store the result in a new edge-service Git repository, and push it to GitHub. The parameters for the initialization are shown in figure 9.2.

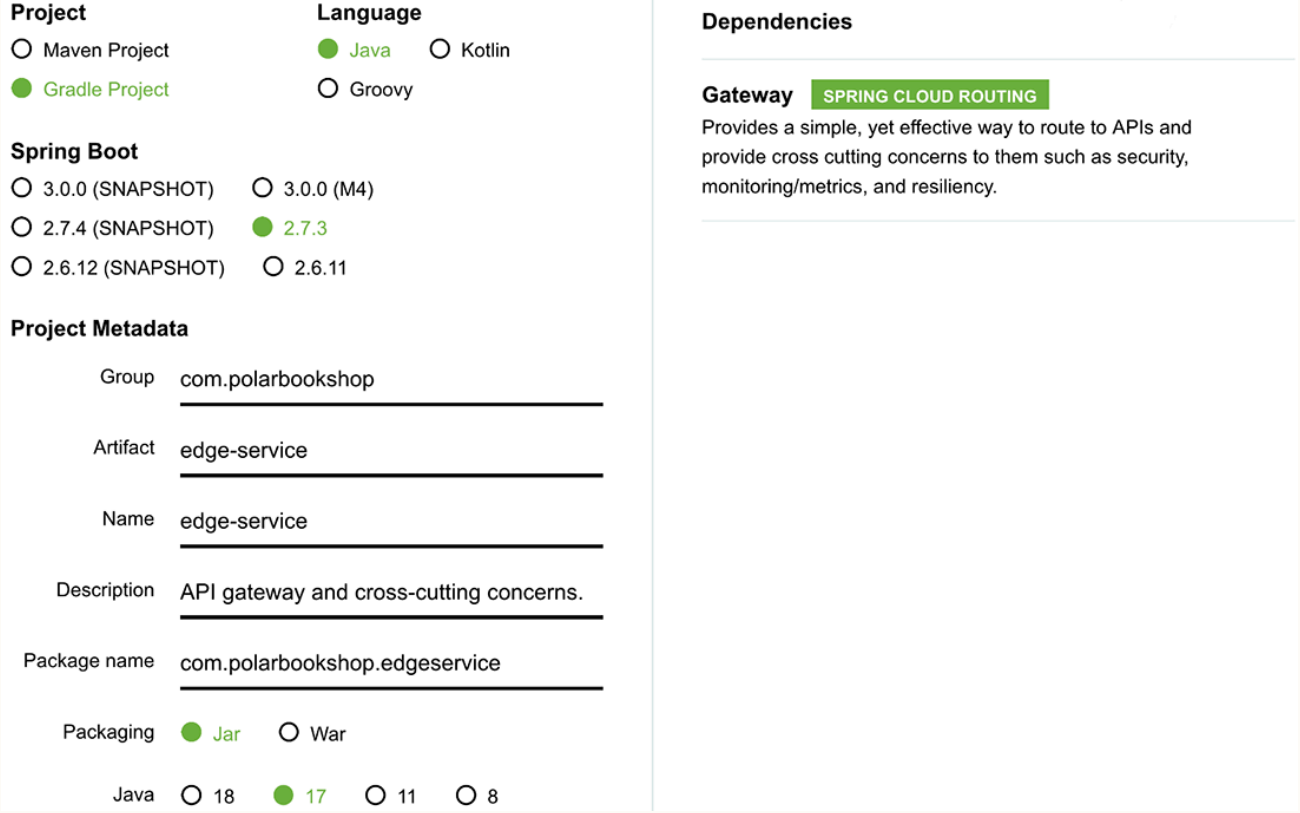


Figure 9.2 The parameters for initializing the Edge Service project

TIP In the begin folder for this chapter, you'll find a curl command you can run in a Terminal window. It downloads a zip file containing all the code you need to get started, without going through the manual generation on the Spring Initializr website.

The dependencies section of the autogenerated build.gradle file looks like this:

```
dependencies {
  implementation 'org.springframework.cloud:spring-cloud-starter-gateway'
  testImplementation 'org.springframework.boot:spring-boot-starter-test'
}
```

These are the main dependencies:

- *Spring Cloud Gateway* (org.springframework.cloud:spring-cloud-starter-gateway)—Provides utilities to route requests to APIs and cross-cutting concerns like resilience, security, and monitoring. It's built on top of the Spring reactive stack.
- *Spring Boot Test* (org.springframework.boot:spring-boot-starter-test)—Provides several libraries and utilities for testing applications, including Spring Test, JUnit, AssertJ, and Mockito. It's automatically included in every Spring Boot project.

At its core, Spring Cloud Gateway is a Spring Boot application. It provides all the convenient features we've been using in the previous chapters, such as auto-configuration, embedded servers, test utilities, externalized configuration, and so on. It's also built on the Spring reactive stack, so you can use the tools and patterns you learned in the previous chapter regarding Spring WebFlux and Reactor. Let's start by configuring the embedded Netty server.

First, rename the application.properties file generated by Spring Initializr (edge-service/src/main/resources) to application.yml. Then open the file and configure the Netty server as you learned in the previous chapter.

Listing 9.1 Configuring Netty server and graceful shutdown

```
server:
  port: 9000                          (1)
  netty:
    connection-timeout: 2s            (2)
    idle-timeout: 15s                 (3)
  shutdown: graceful                  (4)

spring:
  application:
    name: edge-service
  lifecycle:
    timeout-per-shutdown-phase: 15s   (5)
```

(1) The port where the server will accept connections

(2) How long to wait for a TCP connection to be established with the server

(3) How long to wait before closing a TCP connection if no data is transferred

(4) Enables graceful shutdown

(5) Defines a 15 s grace period

The application is set up, so you can move on and start exploring the features of Spring Cloud Gateway.

1.2 Defining routes and predicates

Spring Cloud Gateway provides three main building blocks:

- *Route*—This is identified by a unique ID, a collection of predicates for deciding whether to follow the route, a URI for forwarding the request if the predicates allow, and a collection of filters that are applied either before or after forwarding the request downstream.
- *Predicate*—This matches anything from the HTTP request, including path, host, headers, query parameters, cookies, and body.
- *Filter*—This modifies an HTTP request or response before or after forwarding the request to the downstream service.

Suppose a client sends a request to Spring Cloud Gateway. If the request matches a route through its predicates, the Gateway HandlerMapping will send the request to the Gateway WebHandler, which in turn will run the request through a chain of filters.

There are two filter chains. One chain contains the filters to be run before the request is sent to the downstream service. The other chain is run after sending the request downstream and before forwarding the response. You'll learn about the different types of filters in the next section. Figure 9.3 shows how the routing works in Spring Cloud Gateway.

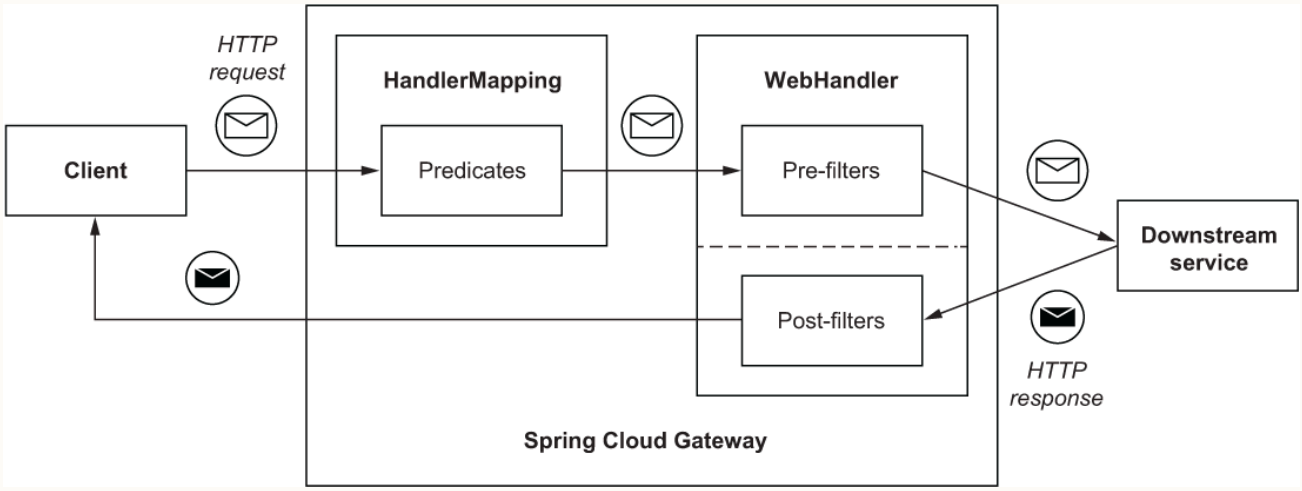


Figure 9.3 Requests are matched against predicates, filtered, and finally forwarded to the downstream service, which replies with a response that goes through another set of filters before being returned to the client.

In the Polar Bookshop system, we have built two applications with APIs that are meant to be accessible from the outside world (public APIs): Catalog Service and Order Service. We can use Edge Service to hide them behind an API gateway. For starters, we need to define the routes.

A minimal route must be configured with a unique ID, a URI where the request should be forwarded, and at least one predicate. Open the application.yml file for the Edge Service project, and configure two routes to Catalog Service and Order Service.

Listing 9.2 Configuring routes to downstream services

```
spring:
  cloud:
    gateway:
      routes:                                    (1)
        - id: catalog-route                      (2)
          uri: ${CATALOG_SERVICE_URL:http://localhost:9001}/books
          predicates:
            - Path=/books/**                     (3)
        - id: order-route
          uri:
↪${ORDER_SERVICE_URL:http://localhost:9002}/orders      (4)
          predicates:
            - Path=/orders/**
```

(1) A list of route definitions

(2) The route ID

(3) The predicate is a path to match

(4) The URI value comes from an environment variable, or else from the default.

Both the routes for Catalog Service and Order Service are matched based on a `Path` predicate. All the incoming requests with a path starting with `/books` will be forwarded to Catalog Service. If the path starts with `/orders`, then Order Service will receive the request. The URIs are computed using the value from an environment variable (`CATALOG_SERVICE_URL` and `ORDER_SERVICE_URL`). If they are not defined, the default value written after the first colon (`:`) symbol will be used. It's an alternative approach compared to how we defined URLs in the previous chapter, based on custom properties; I wanted to show you both options.

The project comes with many different predicates built-in, which you can use in your route configuration to match against any aspect of an HTTP request, including `Cookie`, `Header`, `Host`, `Method`, `Path`, `Query`, and `RemoteAddr`. You can also combine them to form *AND* conditionals. In the previous example, we used the `Path` predicate.

Refer to the official documentation for an extensive list of predicates available in Spring Cloud Gateway: **https://spring.io/projects/spring-cloud-gateway**.

Defining routes with the Java/Kotlin DSL

Spring Cloud Gateway is a very flexible project that lets you configure routes the way that best suits your needs. Here you have configured routes in a property file (application.yml or application.properties), but there's also a DSL available for configuring routes programmatically in Java or Kotlin. Future versions of the project will also implement a feature to fetch the route configuration from a data source using Spring Data.

How you use it is up to you. Putting routes in configuration properties gives you the chance to customize them easily depending on the environment and to update them at runtime without the need to rebuild and redeploy the application. For example, you would get those benefits when using Spring Cloud Config Server. On the other hand, the DSL for Java and Kotlin lets you define more complex routes. Configuration properties allow you to combine different predicates with an *AND* logical operator only. The DSL also enables you to use other logical operators like *OR* and *NOT*.

Let's verify that it works as intended. We'll use Docker to run the downstream services and PostgreSQL, whereas we'll run Edge Service locally on the JVM to make it more efficient to work with, since we are actively implementing the application.

First, we need both Catalog Service and Order Service up and running. From each project's root folder, run `./gradlew bootBuildImage` to package them as container images. Then start them via Docker Compose. Open a Terminal window, navigate to the folder where your docker-compose.yml file is located (polar-deployment/docker), and run the following command:

```
$ docker-compose up -d catalog-service order-service
```

Since both applications depend on PostgreSQL, Docker Compose will also run the PostgreSQL container.

When the downstream services are all up and running, it's time to start Edge Service. From a Terminal window, navigate to the project's root folder (edge-service), and run the following command:

```
$ ./gradlew bootRun
```

The Edge Service application will start accepting requests on port 9000. For the final test, try executing operations on books and orders, but this time through the API gateway (that is, using port 9000 rather than the individual ports to which Catalog Service and Order Service are listening). They should return a 200 OK response:

```
$ http :9000/books
$ http :9000/orders
```

The result is the same as if you called Catalog Service and Order Service directly, but you only need to know one hostname and port this time. When you are done testing the application, stop its execution with Ctrl-C. Then terminate all the containers with Docker Compose:

```
$ docker-compose down
```

Under the hood, Edge Service uses Netty's HTTP client to forward requests to downstream services. As extensively discussed in the previous chapter, whenever an application calls an external service, it's essential to configure a timeout to make it resilient to interprocess communication failures. Spring Cloud Gateway provides dedicated properties to configure the HTTP client timeouts.

Open the Edge Service application.yml file once again, and define values for the connection timeout (the time limit for a connection to be established with the downstream service) and for the response timeout (the time limit for receiving a response).

Listing 9.3 Configuring timeouts for the gateway HTTP client

```
spring:
  cloud:
    gateway:
      httpclient:                  (1)
        connect-timeout: 2000      (2)
        response-timeout: 5s       (3)
```

(1) Configuration properties for the HTTP client

❷ Time limit for a connection to be established (in ms)

❸ Time limit for a response to be received (Duration)

By default, the Netty HTTP client used by Spring Cloud Gateway is configured with an *elastic* connection pool to increase the number of concurrent connections dynamically as the workload increases. Depending on the number of requests your system receives simultaneously, you might want to switch to a *fixed* connection pool so you have more control over the number of connections. You can configure the Netty connection pool in Spring Cloud Gateway through the spring.cloud.gateway.httpclient.pool property group in the application.yml file.

Listing 9.4 Configuring the connection pool for the gateway HTTP client

```
spring:
  cloud:
    gateway:
      httpclient:
        connect-timeout: 5000
        response-timeout: 5s
        pool:
          type: elastic        ❶
          max-idle-time: 15s   ❷
          max-life-time: 60s   ❸
```

❶ Type of connection pool (elastic, fixed, or disabled)

❷ Idle time after which the communication channel will be closed

❸ Time after which the communication channel will be closed

You can refer to the official Reactor Netty documentation for more details about how the connection pool works, what configurations are available, and tips on what values to use based on specific scenarios (**https://projectreactor.io/docs**).

In the next section, we'll start implementing something more interesting than merely forwarding requests—we'll look at the power of Spring Cloud Gateway filters.

11.3 Processing requests and responses through filters

Routes and predicates alone make the application act as a proxy, but it's filters that make Spring Cloud Gateway really powerful.

Filters can run before forwarding incoming requests to a downstream application (*pre-filters*). They can be used for:

- Manipulating the request headers
- Applying rate limiting and circuit breaking
- Defining retries and timeouts for the proxied request
- Triggering an authentication flow with OAuth2 and OpenID Connect

Other filters can apply to outgoing responses after they are received from the downstream application and before sending them back to the client (*post-filters*). They can be used for:

- Setting security headers
- Manipulating the response body to remove sensitive information

Spring Cloud Gateway comes bundled with many filters that you can use to perform different actions, including adding headers to a request, configuring a circuit breaker, saving the web session, retrying the request on failure, or activating a rate limiter.

In the previous chapter, you learned how to use the retry pattern to improve application resilience. You'll now learn how to apply it as a default filter for all `GET` requests going through the routes defined in the gateway.

USING THE RETRY FILTER

You can define default filters in the application.yml file located under src/main/ resources. One of the filters provided by Spring Cloud Gateway is the `Retry` filter. The configuration is similar to what we did in chapter 8.

Let's define a maximum of three retry attempts for all `GET` requests whenever the error is in the `5xx` range (`SERVER_ERROR`). We don't want to retry requests when the error is in the `4xx` range. For example, if the result is a `404` response, it doesn't make sense to retry the request. We can also list the exceptions for which a retry should be attempted, such as `IOException` and `TimeoutException`.

By now, you know that you shouldn't keep retrying requests one after the other. You should use a backoff strategy instead. By default, the delay is computed using the formula firstBackoff * (factor ^ n). If you set the basedOnPreviousValue parameter to true, the formula will be prevBackoff * factor.

Listing 9.5 Applying the retry filter to all routes

```
spring:
  cloud:
    gateway:
      default-filters:                          (1)
        - name: Retry                           (2)
          args:
            retries: 3                          (3)
            methods: GET                        (4)
            series: SERVER_ERROR                (5)
            exceptions: java.io.IOException,
            ➥java.util.concurrent.TimeoutException   (6)
            backoff:                            (7)
              firstBackoff: 50ms
              maxBackOff: 500ms
              factor: 2
              basedOnPreviousValue: false
```

(1) A list of default filters

(2) The name of the filter

(3) Maximum of 3 retry attempts

(4) Retries only GET requests

(5) Retries only when 5XX errors

(6) Retries only when the given exceptions are thrown

(7) Retries with a delay computed as "firstBackoff * (factor ^ n)"

The retry pattern is useful when a downstream service is momentarily unavailable. But what if it stays down for more than a few instants? At that point we could stop forwarding requests to it until we're sure that it's back. Continuing to send requests won't be

beneficial for the caller or the callee. In that scenario, the circuit breaker pattern comes in handy. That's the topic of the next section.

9.2 Fault tolerance with Spring Cloud Circuit Breaker and Resilience4J

As you know, resilience is a critical property of cloud native applications. One of the principles for achieving resilience is blocking a failure from cascading and affecting other components. Consider a distributed system where application X depends on application Y. If application Y fails, will application X fail, too? A circuit breaker can block a failure in one component from propagating to the others depending on it, protecting the rest of the system. That is accomplished by temporarily stopping communication with the faulty component until it recovers. This pattern comes from electrical systems, for which the circuit is physically opened to break the electrical connection and avoid destroying the entire house when a part of the system fails due to current overload.

In the world of distributed systems, you can establish circuit breakers at the integration points between components. Think about Edge Service and Catalog Service. In a typical scenario, the circuit is *closed*, meaning that the two services can interact over the network. For each server error response returned by Catalog Service, the circuit breaker in Edge Service would register the failure. When the number of failures exceeds a certain threshold, the circuit breaker trips, and the circuit transitions to *open*.

While the circuit is open, communications between Edge Service and Catalog Service are not allowed. Any request that should be forwarded to Catalog Service will fail right away. In this state, either an error is returned to the client, or fallback logic is executed. After an appropriate amount of time to permit the system to recover, the circuit breaker transitions to a *half-open* state, allowing the next call to Catalog Service to go through. That is an exploratory phase to check if there are still issues in contacting the downstream service. If the call succeeds, the circuit breaker is reset and transitions to *closed*. Otherwise it goes back to being *open*. Figure 9.4 shows how a circuit breaker changes state.

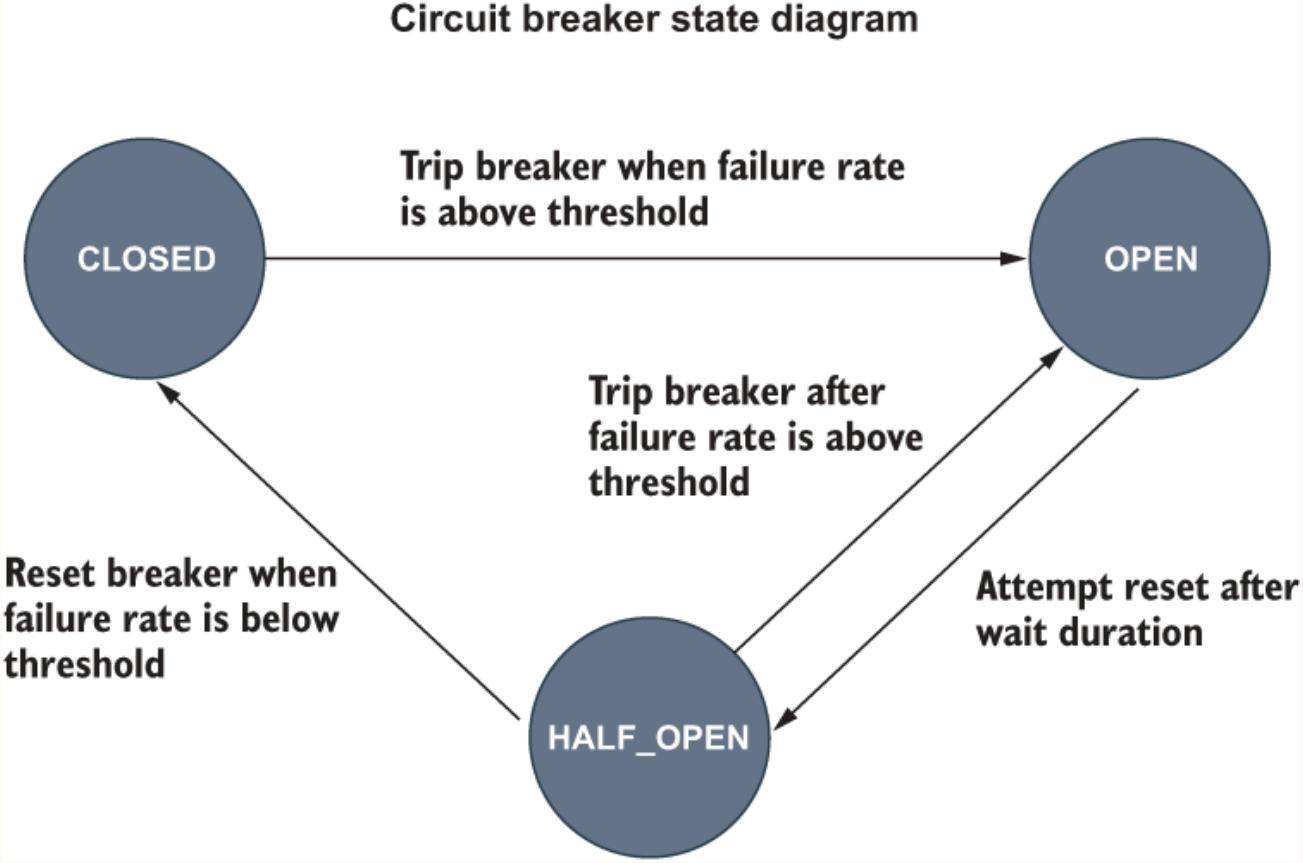


Figure 9.4 A circuit breaker ensures fault tolerance when a downstream service exceeds the maximum number of failures allowed by blocking any communication between upstream and downstream services. The logic is based on three states: closed, open, and half-open.

Unlike with retries, when the circuit breaker trips, no calls to the downstream service are allowed anymore. Like with retries, the circuit breaker's behavior depends on a threshold and a timeout, and it lets you define a fallback method to call. The goal of resilience is to keep the system available to users, even in the face of failures. In the worst-case scenario, like when a circuit breaker trips, you should guarantee a graceful degradation. You can adopt different strategies for the fallback method. For example, you might decide to return a default value or the last available value from a cache, in case of a GET request.

The Spring Cloud Circuit Breaker project provides an abstraction for defining circuit breakers in a Spring application. You can choose between reactive and non-reactive implementations based on Resilience4J (**https://resilience4j.readme.io**). Netflix Hystrix was the popular choice for microservices architectures, but it entered maintenance mode back in 2018. After that, Resilience4J became the preferred choice because it provides the same features offered by Hystrix and more.

Spring Cloud Gateway integrates natively with Spring Cloud Circuit Breaker, providing you with a CircuitBreaker gateway filter that you can use to protect the interactions

with all downstream services. In the following sections, you'll configure a circuit breaker for the routes to Catalog Service and Order Service from Edge Service.

12.2.1 Introducing circuit breakers with Spring Cloud Circuit Breaker

To use Spring Cloud Circuit Breaker in Spring Cloud Gateway, you need to add a dependency to the specific implementation you'd like to use. In this case, we'll use the Resilience4J reactive version. Go ahead and add the new dependency in the build.gradle file for the Edge Service project (edge-service). Remember to refresh or reimport the Gradle dependencies after the new addition.

Listing 9.6 Adding dependency for Spring Cloud Circuit Breaker

```
dependencies {
  ...
  implementation 'org.springframework.cloud:
  ➥spring-cloud-starter-circuitbreaker-reactor-resilience4j'
}
```

The CircuitBreaker filter in Spring Cloud Gateway relies on Spring Cloud Circuit Breaker to wrap a route. As with the Retry filter, you can choose to apply it to specific routes or define it as a default filter. Let's go with the first option. You can also specify an optional fallback URI to handle the request when the circuit is in an open state. In this example (application.yml), both routes will be configured with a CircuitBreaker filter, but only catalog-route will have a fallbackUri value so that I can show you both scenarios.

Listing 9.7 Configuring circuit breakers for the gateway routes

```
spring:
  cloud:
    gateway:
      routes:
        - id: catalog-route
          uri: ${CATALOG_SERVICE_URL:http://localhost:9001}/books
          predicates:
            - Path=/books/**
          filters:
            - name: CircuitBreaker                    (1)
              args:
                name: catalogCircuitBreaker           (2)
```

```
          fallbackUri: forward:/catalog-fallback      ❸
      - id: order-route
        uri: ${ORDER_SERVICE_URL:http://localhost:9002}/orders
        predicates:
          - Path=/orders/**
        filters:
          - name: CircuitBreaker                      ❹
            args:
              name: orderCircuitBreaker
```

❶ Name of the filter

❷ Name of the circuit breaker

❸ Forwards request to this URI when the circuit is open

❹ No fallback defined for this circuit breaker.

The next step is configuring the circuit breaker.

12.2 Configuring a circuit breaker with Resilience4J

After defining which routes you want to apply the CircuitBreaker filter to, you need to configure the circuit breakers themselves. As often in Spring Boot, you have two main choices. You can configure circuit breakers through the properties provided by Resilience4J or via a Customizer bean. Since we're using the reactive version of Resilience4J, the specific configuration bean would be of type Customizer<ReactiveResilience4JCircuitBreakerFactory>.

Either way, you can choose to define a specific configuration for each circuit breaker you used in your application.yml file (catalogCircuitBreaker and orderCircuitBreaker in our case) or declare some defaults that will be applied to all of them.

For the current example, we can define circuit breakers to consider a window of 20 calls (slidingWindowSize). Each new call will make the window move, dropping the oldest registered call. When at least 50% of the calls in the window have produced an error (failureRateThreshold), the circuit breaker will trip, and the circuit will enter the open state. After 15 seconds (waitDurationInOpenState), the circuit will be allowed to transition to a half-open state in which 5 calls are permitted (permittedNumberOfCallsInHalfOpenState). If at least 50% of them result in an error, the

circuit will go back to the open state. Otherwise, the circuit breaker will trip to the closed state.

On to the code. In the Edge Service project (edge-service), at the end of the application.yml file, define a default configuration for all Resilience4J circuit breakers.

Listing 9.8 Configuring circuit breaker and time limiter

```
resilience4j:
  circuitbreaker:
    configs:
      default:                                  (1)
        slidingWindowSize: 20                   (2)
        permittedNumberOfCallsInHalfOpenState: 5  (3)
        failureRateThreshold: 50                (4)
        waitDurationInOpenState: 15000          (5)
  timelimiter:
    configs:
      default:                                  (6)
        timeoutDuration: 5s                     (7)
```

(1) Default configuration bean for all circuit breakers

(2) The size of the sliding window used to record the outcome of calls when the circuit is closed

(3) Number of permitted calls when the circuit is half-open

(4) When the failure rate is above the threshold, the circuit becomes open.

(5) Waiting time before moving from open to half-open (ms)

(6) Default configuration bean for all time limiters

(7) Configures a timeout (seconds)

We configure both the circuit breaker and a time limiter, a required component when using the Resilience4J implementation of Spring Cloud Circuit Breaker. The timeout configured via Resilience4J will take precedence over the response timeout we defined in the previous section for the Netty HTTP client (spring.cloud.gateway.httpclient.response-timeout).

When a circuit breaker switches to the open state, we'll want at least to degrade the service level gracefully and make the user experience as pleasant as possible. I'll show you how to do that in the next section.

12.3 Defining fallback REST APIs with Spring WebFlux

When we added the CircuitBreaker filter to catalog-route, we defined a value for the fallbackUri property to forward the requests to the /catalog-fallback endpoint when the circuit is in an open state. Since the Retry filter is also applied to that route, the fallback endpoint will be invoked even when all retry attempts fail for a given request. It's time to define that endpoint.

As I mentioned in previous chapters, Spring supports defining REST endpoints either using @RestController classes or router functions. Let's use the functional way of declaring the fallback endpoints.

In a new com.polarbookshop.edgeservice.web package in the Edge Service project, create a new WebEndpoints class. Functional endpoints in Spring WebFlux are defined as routes in a RouterFunction<ServerResponse> bean, using the fluent API provided by RouterFunctions. For each route, you need to define the endpoint URL, a method, and a handler.

Listing 9.9 Fallback endpoints for when the Catalog Service is down

```
package com.polarbookshop.edgeservice.web;

import reactor.core.publisher.Mono;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpStatus;
import org.springframework.web.reactive.function.server.RouterFunction;
import org.springframework.web.reactive.function.server.RouterFunctions;
import org.springframework.web.reactive.function.server.ServerResponse;

@Configuration
public class WebEndpoints {

  @Bean                                                    (1)
  public RouterFunction<ServerResponse> routerFunction() {
    return RouterFunctions.route()                         (2)
      .GET("/catalog-fallback", request ->                 (3)
        ServerResponse.ok().body(Mono.just(""), String.class))
```

```
        .POST("/catalog-fallback", request ->                    ❹
          ServerResponse.status(HttpStatus.SERVICE_UNAVAILABLE).build())
        .build();                                                ❺
    }
}
```

❶ Functional REST endpoints are defined in a bean.

❷ Offers a fluent API to build routes

❸ Fallback response used to handle the GET endpoint

❹ Fallback response used to handle the POST endpoint

❺ Builds the functional endpoints

For simplicity, the fallback for GET requests returns an empty string, whereas the fallback for POST requests returns an HTTP 503 error. In a real scenario, you might want to adopt different fallback strategies depending on the context, including throwing a custom exception to be handled from the client or returning the last value saved in the cache for the original request.

So far, we have used retries, timeouts, circuit breakers, and failovers (fallbacks). In the next section, I'll expand on how we can work with all those resilience patterns together.

2.4 Combining circuit breakers, retries, and time limiters

When you combine multiple resilience patterns, the sequence in which they are applied is fundamental. Spring Cloud Gateway takes care of applying the TimeLimiter first (or the timeout on the HTTP client), then the CircuitBreaker filter, and finally Retry. Figure 9.5 shows how these patterns work together to increase the application's resilience.

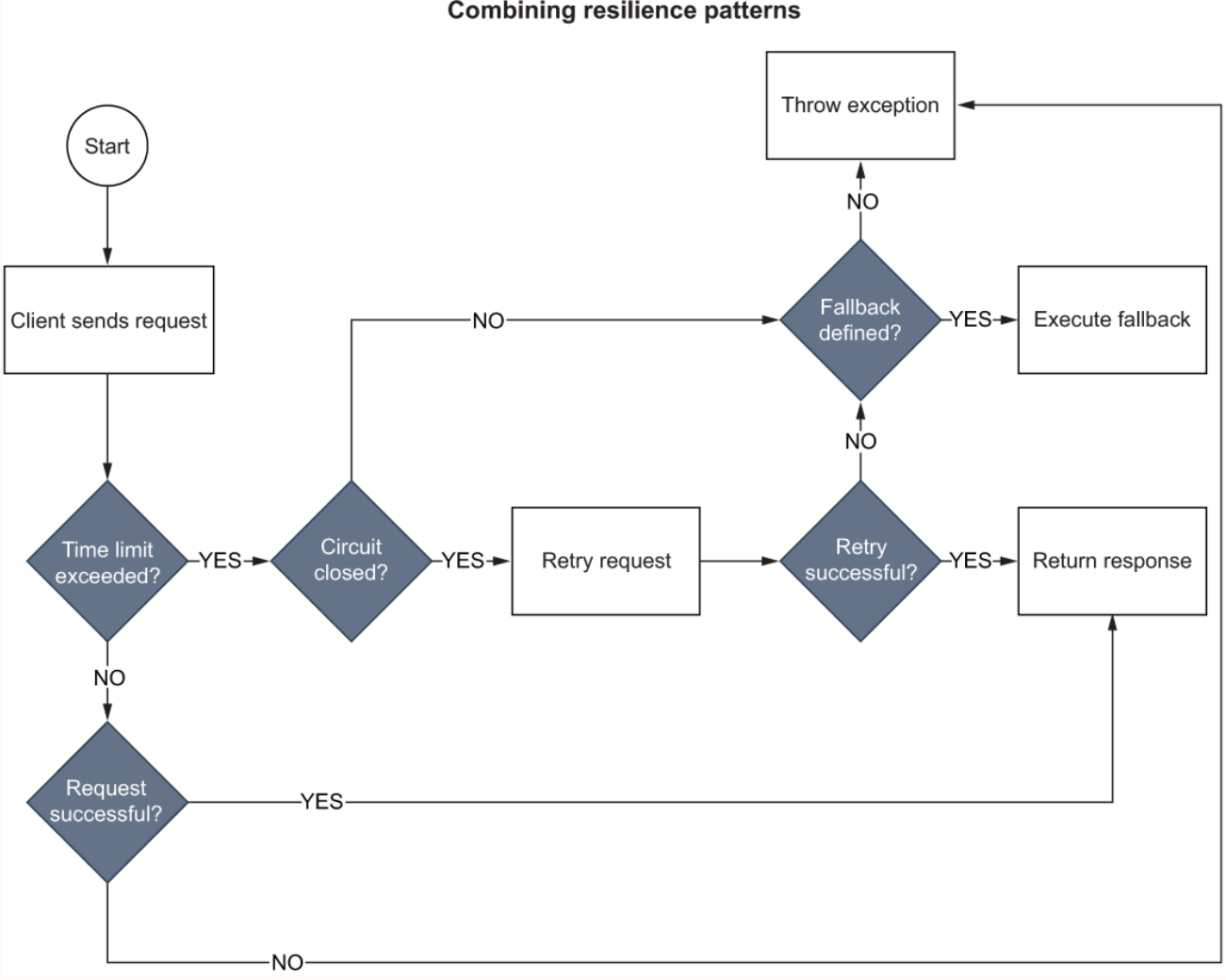


Figure 9.5 When multiple resilience patterns are implemented, they are applied in a specific sequence.

You can verify the result of applying these patterns to Edge Service by using a tool like Apache Benchmark (**https://httpd.apache.org/docs/2.4/programs/ab.html**). If you're using macOS or Linux, you might have this tool already installed. Otherwise, you can follow the instructions on the official website and install it.

Make sure both Catalog Service and Order Service are not running so that you can test circuit breakers in a failure scenario. Then enable debug logging for Resilience4J so you can follow the state transitions of the circuit breaker. At the end of the application.yml file in your Edge Service project, add the following configuration.

Listing 9.10 Enabling debug logging for Resilience4J

```
logging:
  level:
    io.github.resilience4j: DEBUG
```

Next, build and run Edge Service (./gradlew bootRun). Since no downstream services are running (if they are, you should stop them), all the requests sent to them from Edge Service will result in errors. Let's see what happens if we run 21 sequential POST requests (-n 21 -c 1 -m POST) to the /orders endpoint. Remember that POST requests have no retry configuration, and order-route has no fallback, so the result will only be affected by the timeout and circuit breaker:

```
$ ab -n 21 -c 1 -m POST http://localhost:9000/orders
```

From the ab output, you can see that all the requests returned an error:

```
Complete requests: 21
Non-2xx responses: 21
```

The circuit breaker is configured to trip to the open state when at least 50% of the calls in a 20-sized time window fails. Since you have just started the application, the circuit will transition to the open state after 20 requests. In the application logs, you can analyze how the requests have been handled. All the requests failed, so the circuit breaker registers an ERROR event for each of them:

```
Event ERROR published: CircuitBreaker 'orderCircuitBreaker'
  recorded an error.
```

At the 20th request, a FAILURE_RATE_EXCEEDED event is recorded because it exceeded the failure threshold. That will result in a STATE_TRANSITION event that will open the circuit:

```
Event ERROR published: CircuitBreaker 'orderCircuitBreaker'
  recorded an error.
Event FAILURE_RATE_EXCEEDED published: CircuitBreaker 'orderCircuitBreaker'
  exceeded failure rate threshold.
Event STATE_TRANSITION published: CircuitBreaker 'orderCircuitBreaker'
  changed state from CLOSED to OPEN
```

The 21st request will not even try contacting Order Service: the circuit is open, so it cannot go through. A NOT_PERMITTED event is registered to signal why the request failed:

```
Event NOT_PERMITTED published: CircuitBreaker 'orderCircuitBreaker'
  recorded a call which was not permitted.
```

NOTE Monitoring the status of circuit breakers in production is a critical task. In chapter 13, I'll show you how to export that information as Prometheus metrics that you can visualize in a Grafana dashboard instead of checking the logs. In the meantime, for a more visual explanation, feel free to watch my "Spring Cloud Gateway: Resilience, Security, and Observability" session on circuit breakers at Spring I/O, 2022 (http://mng.bz/z55A).

Now let's see what happens when we call a GET endpoint for which both retries and fallback have been configured. Before proceeding, rerun the application so you can start with a clear circuit breaker state (./gradlew bootRun). Then run the following command:

```
$ ab -n 21 -c 1 -m GET http://localhost:9000/books
```

If you check the application logs, you'll see how the circuit breaker behaves precisely like before: 20 allowed requests (closed circuit), followed by a non-permitted request (open circuit). However, the result of the previous command shows 21 requests completed with no errors:

```
Complete requests: 21
Failed requests: 0
```

This time, all requests have been forwarded to the fallback endpoint, so the client didn't experience any errors.

We configured the Retry filter to be triggered when an IOException or TimeoutException occurs. In this case, since the downstream service is not running, the exception thrown is of type ConnectException, so the request is conveniently not retried, which allowed me to show you the combined behavior of circuit breakers and fallbacks without retries.

So far we have looked at patterns that make the interactions between Edge Service and the downstream applications more resilient. What about the entry point of the system? The next section will introduce rate limiters, which will control the request flow coming into the system through the Edge Service application. Before proceeding, stop the application's execution with Ctrl-C.

9.3 Request rate limiting with Spring Cloud Gateway and Redis

Rate limiting is a pattern used to control the rate of traffic sent to or received from an application, helping to make your system more resilient and robust. In the context of HTTP interactions, you can apply this pattern to control outgoing or incoming network traffic using client-side and server-side rate limiters, respectively.

Client-side rate limiters are for constraining the number of requests sent to a downstream service in a given period. It's a useful pattern to adopt when third-party organizations like cloud providers manage and offer the downstream service. You'll want to avoid incurring extra costs for having sent more requests than are allowed by your subscription. In the case of pay-per-use services, this helps prevent unexpected expenses.

If the downstream service belongs to your system, you might use a rate limiter to avoid causing DoS problems for yourself. In this case, though, a *bulkhead* pattern (or *concurrent request limiter*) would be a better fit, setting constraints on how many concurrent requests are allowed and queuing up the blocked ones. Even better is an adaptive bulkhead, for which the concurrency limits are dynamically updated by an algorithm to better adapt to the elasticity of cloud infrastructure.

Server-side rate limiters are for constraining the number of requests received by an upstream service (or client) in a given period. This pattern is handy when implemented in an API gateway to protect the whole system from overloading or from DoS attacks. When the number of users increases, the system should scale in a resilient way, ensuring an acceptable quality of service for all users. Sudden increases in user traffic are expected, and they are usually initially addressed by adding more resources to the infrastructure or more application instances. Over time, though, they can become a problem and even lead to service outages. Server-side rate limiters help with that.

When a user has exceeded the number of allowed requests in a specific time window, all the extra requests are rejected with an HTTP 429 - Too Many Requests status. The limit is applied according to a given strategy. For example, you can limit requests per session, per IP address, per user, or per tenant. The overall goal is to keep the system available for all users in case of adversity. That is the definition of resilience. This pattern is also handy for offering services to users depending on their subscription tiers. For example, you might define different rate limits for basic, premium, and enterprise users.

Resilience4J supports the client-side rate limiter and bulkhead patterns for both reactive and non-reactive applications. Spring Cloud Gateway supports the server-side rate lim-

iter pattern, and this section will show you how to use it for Edge Service by using Spring Cloud Gateway and Spring Data Redis Reactive. Let's start with setting up a Redis container.

3.1 Running Redis as a container

Imagine you want to limit access to your API so that each user can only perform 10 requests per second. Implementing such a requirement would require a storage mechanism to track the number of requests each user performs every second. When the limit is reached, the following requests should be rejected. When the second is over, each user can perform 10 more requests within the next second. The data used by the rate-limiting algorithm is small and temporary, so you might think of saving it in memory inside the application itself.

However, that would make the application stateful and lead to errors, since each application instance would limit requests based on a partial data set. It would mean letting users perform 10 requests per second per instance rather than overall, because each instance would only keep track of its own incoming requests. The solution is to use a dedicated data service to store the rate-limiting state and make it available to all the application replicas. Enter Redis.

Redis (**https://redis.com**) is an in-memory store that is commonly used as a cache, message broker, or database. In Edge Service, we'll use it as the data service backing the request rate limiter implementation provided by Spring Cloud Gateway. The Spring Data Redis Reactive project provides the integration between a Spring Boot application and Redis.

Let's first define a Redis container. Open the docker-compose.yml file you created in your polar-deployment repository. (If you haven't followed along with the examples, you can use Chapter09/09-begin/polar-deployment/docker/docker-compose.yml from the source code accompanying the book as a starting point.) Then add a new service definition using the Redis official image, and expose it through port 6379.

Listing 9.11 Defining a Redis container

```
version: "3.8"
services:
  ...
  polar-redis:
    image: "redis:7.0"              (1)
    container_name: "polar-redis"
```

```
    ports:
      - 6379:6379                    ❷
```

❶ Uses Redis 7.0

❷ Exposes Redis through port 6379

Next, open a Terminal window, navigate to the folder where your docker-compose.yml file is located, and run the following command to start a Redis container:

```
$ docker-compose up -d polar-redis
```

In the following section, you'll configure the Redis integration with Edge Service.

3.2 Integrating Spring with Redis

The Spring Data project has modules supporting several database options. In the previous chapters, we worked with Spring Data JDBC and Spring Data R2DBC to use relational databases. Now we'll use Spring Data Redis, which provides support for this in-memory, non-relational data store. Both imperative and reactive applications are supported.

First we need to add a new dependency on Spring Data Redis Reactive in the build.gradle file of the Edge Service project (edge-service). Remember to refresh or reimport the Gradle dependencies after the new addition.

Listing 9.12 Adding dependency for Spring Data Redis Reactive

```
dependencies {
  ...
  implementation
  ➥'org.springframework.boot:spring-boot-starter-data-redis-reactive'
}
```

Then, in the application.yml file, configure the Redis integration through the properties provided by Spring Boot. Besides spring.redis.host and spring.redis.port for defining where to reach Redis, you can also specify connection and read timeouts using spring.redis.connect-timeout and spring.redis.timeout respectively.

Listing 9.13 Configuring the Redis integration

```
spring:
  redis:
    connect-timeout: 2s    ❶
    host: localhost        ❷
    port: 6379             ❸
    timeout: 1s            ❹
```

❶ Time limit for a connection to be established

❷ Default Redis host

❸ Default Redis port

❹ Time limit for a response to be received

In the next section, you'll see how to use Redis to back the RequestRateLimiter gateway filter that provides server-side rate limiting support.

3.3 Configuring a request rate limiter

Depending on the requirements, you can configure the RequestRateLimiter filter for specific routes or as a default filter. In this case we'll configure it as a default filter so that it's applied to all routes, current and future.

The implementation of RequestRateLimiter on Redis is based on the *token bucket algorithm*. Each user is assigned a bucket inside which tokens are dripped over time at a specific rate (the *replenish rate*). Each bucket has a maximum capacity (the *burst capacity*). When a user makes a request, a token is removed from its bucket. When there are no more tokens left, the request is not permitted, and the user will have to wait until more tokens are dripped into its bucket.

NOTE If you want to know more about the token bucket algorithm, I recommend reading Paul Tarjan's "Scaling your API with Rate Limiters" article about how they use it to implement rate limiters at Stripe (**https://stripe.com/blog/rate-limiters**).

For this example, let's configure the algorithm so that each request costs 1 token (redis-rate-limiter.requestedTokens). Tokens are dripped in the bucket following the configured replenish rate (redis-rate-limiter.replenishRate), which we'll set as 10 tokens per second. Sometimes there might be spikes, resulting in a larger number of requests than usual. You can allow temporary bursts by defining a larger capacity for

the bucket (redis-rate-limiter.burstCapacity), such as 20. This means that when a spike occurs, up to 20 requests are allowed per second. Since the replenish rate is lower than the burst capacity, subsequent bursts are not allowed. If two spikes happen sequentially, only the first one will succeed, while the second will result in some requests being dropped with an HTTP 429 - Too Many Requests response. The resulting configuration in the application.yml file is shown in the following listing.

Listing 9.14 Configuring a request rate limiter as a gateway filter

```
spring:
  cloud:
    gateway:
      default-filters:
        name: RequestRateLimiter
          args:
            redis-rate-limiter:
              replenishRate: 10     (1)
              burstCapacity: 20     (2)
              requestedTokens: 1    (3)
```

(1) Number of tokens dripped in the bucket each second

(2) Allows request bursts of up to 20 requests

(3) How many tokens a request costs

There's no general rule to follow in coming up with good numbers for the request rate limiter. You should start with your application requirements and go with a trial and error approach: analyze your production traffic, tune the configuration, and do this all over again until you achieve a setup that keeps your system available while not affecting the user experience badly. Even after that, you should keep monitoring the status of your rate limiters, since things can change in the future.

Spring Cloud Gateway relies on Redis to keep track of the number of requests happening each second. By default, each user is assigned a bucket. However, we haven't introduced an authentication mechanism yet, so we'll use a single bucket for all requests until we address the security concerns in chapters 11 and 12.

NOTE What happens if Redis becomes unavailable? Spring Cloud Gateway has been built with resilience in mind, so it will keep its service level, but the rate limiters would be disabled until Redis is up and running again.

The RequestRateLimiter filter relies on a KeyResolver bean to determine which bucket to use for each request. By default, it uses the currently authenticated user in Spring Security. Until we add security to Edge Service, we'll define a custom KeyResolver bean and make it return a constant value (for example, anonymous) so that all requests will be mapped to the same bucket.

In your Edge Service project, create a RateLimiterConfig class in a new com.polarbookshop.edgeservice.config package, and declare a KeyResolver bean, implementing a strategy to return a constant key.

Listing 9.15 Defining a strategy to resolve the bucket to use per request

```
package com.polarbookshop.edgeservice.config;

import reactor.core.publisher.Mono;
import org.springframework.cloud.gateway.filter.ratelimit.KeyResolver;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RateLimiterConfig {

  @Bean
  public KeyResolver keyResolver() {
    return exchange -> Mono.just("anonymous");     (1)
  }
}
```

(1) Rate limiting is applied to requests using a constant key.

Spring Cloud Gateway is configured to append headers with details about rate-limiting to each HTTP response, which we can use to verify its behavior. Rebuild and run Edge Service (./gradlew bootRun), and then try calling one of the endpoints.

```
$ http :9000/books
```

The response body depends on whether Catalog Service is running or not, but that doesn't matter in this example. The interesting aspect to notice is the HTTP headers of the response. They show the rate limiter's configuration and the number of remaining requests allowed within the time window (1 second):

```
HTTP/1.1 200 OK
Content-Type: application/json
X-RateLimit-Burst-Capacity: 20
X-RateLimit-Remaining: 19
X-RateLimit-Replenish-Rate: 10
X-RateLimit-Requested-Tokens: 1
```

You might not want to expose this information to clients in cases where the information could help bad actors craft attacks against your system. Or you might need different header names. Either way, you can use the spring.cloud.gateway.redis-rate-limiter property group to configure that behavior. When you're done testing the application, stop it with Ctrl-C.

NOTE When the rate limiter pattern is combined with other patterns like time limiters, circuit breakers, and retries, the rate limiter is applied first. If a user's request exceeds the rate limit, it is rejected right away.

Redis is an efficient data store ensuring fast data access, high availability, and resilience. In this section, we used it to provide storage for the rate limiters, and the next section will show you how to use it in another common scenario: session management.

9.4 Distributed session management with Redis

In the previous chapters, I often highlighted how cloud native applications should be stateless. We scale them in and out, and if they weren't stateless, we would lose the state every time an instance is shut down. Some state needs to be saved, or the applications would probably be useless. For example, Catalog Service and Order Service are stateless, but they rely on a stateful service (the PostgreSQL database) to permanently store the data about books and orders. Even if the applications are shut down, the data will survive and be available to all the application instances.

Edge Service is not dealing with any business entities it needs to store, but it still needs a stateful service (Redis) to store the state related to the RequestRateLimiter filter. When Edge Service is replicated, it's important to keep track of how many requests are left before exceeding the threshold. Using Redis, the rate limiter functionality is guaranteed consistently and safely.

Furthermore, in chapter 11 you'll expand Edge Service to add authentication and authorization. Since it's the entry point to the Polar Bookshop system, it makes sense to authenticate the user there. Data about the authenticated session will have to be saved

outside the application for the same reason as the rate limiter information is. If it wasn't, the user might have to authenticate themselves every time a request hits a different Edge Service instance.

The general idea is to keep the applications stateless and use data services for storing the state. As you learned in chapter 5, data services need to guarantee high availability, replication, and durability. In your local environment, you can ignore that aspect, but in production you'll rely on the data services offered by cloud providers, both for PostgreSQL and Redis.

The following section will cover how you can work with Spring Session Data Redis to establish distributed session management.

9.4.1 Handling sessions with Spring Session Data Redis

Spring provides session management features with the Spring Session project. By default, session data is stored in memory, but that's not feasible in a cloud native application. You want to keep it in an external service so that the data survives the application shutdown. Another fundamental reason for using a distributed session store is that you usually have multiple instances of a given application. You'll want them to access the same session data to provide a seamless experience to the user.

Redis is a popular option for session management, and it's supported by Spring Session Data Redis. Furthermore, you have already set it up for the rate limiters. You can add it to Edge Service with minimal configuration.

First you need to add a new dependency on Spring Session Data Redis to the build.gradle file for the Edge Service project. You can also add the Testcontainers library so you can use a lightweight Redis container when writing integration tests. Remember to refresh and reimport the Gradle dependencies after the new addition.

Listing 9.16 Adding dependency for Spring Session and Testcontainers

```
ext {
  ...
  set('testcontainersVersion', "1.17.3")
}

dependencies {
  ...
  implementation 'org.springframework.session:spring-session-data-redis'
  testImplementation 'org.testcontainers:junit-jupiter'
```

```
}

dependencyManagement {
  imports {
    ...
    mavenBom
    "org.testcontainers:testcontainers-bom:${testcontainersVersion}"
  }
}
```

Next, you need to instruct Spring Boot to use Redis for session management (spring.session.store-type) and define a unique namespace to prefix all session data coming from Edge Service (spring.session.redis.namespace). You can also define a timeout for the session (spring.session.timeout). If you don't specify a timeout, the default is 30 minutes.

Configure Spring Session in the application.yml file as follows.

Listing 9.17 Configuring Spring Session to store data in Redis

```
spring:
  session:
    store-type: redis
    timeout: 10m
    redis:
      namespace: polar:edge
```

Managing web sessions in a gateway requires some additional care to ensure you save the right state at the right time. In this example, we want the session to be saved in Redis before forwarding a request downstream. How can we do that? If you were thinking about whether there's a gateway filter for it, you would be right!

In the application.yml file for the Edge Service project, add SaveSession as a default filter to instruct Spring Cloud Gateway to always save the web session before forwarding requests downstream.

Listing 9.18 Configuring the gateway to save the session data

```
spring:
  cloud:
    gateway:
```

```
      default-filters:
        - SaveSession      (1)
```

(1) Ensures the session data is saved before forwarding a request downstream

That's a critical point when Spring Session is combined with Spring Security. Chapters 11 and 12 will cover more details about session management. For now, let's set up an integration test to verify the Spring context in Edge Service loads correctly, including the integration with Redis.

The approach we'll use is similar to the one we used to define PostgreSQL test containers in the previous chapter. Let's extend the existing EdgeServiceApplicationTests class generated by Spring Initializr and configure a Redis test container. For this example, it's enough to verify that the Spring context loads correctly when Redis is used for storing web session-related data.

Listing 9.19 Using a Redis container to test the Spring context loading

```
package com.polarbookshop.edgeservice;

import org.junit.jupiter.api.Test;
import org.testcontainers.containers.GenericContainer;
import org.testcontainers.junit.jupiter.Container;
import org.testcontainers.junit.jupiter.Testcontainers;
import org.testcontainers.utility.DockerImageName;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.DynamicPropertyRegistry;
import org.springframework.test.context.DynamicPropertySource;

@SpringBootTest(                                    (1)
  webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT
)
@Testcontainers                                     (2)
class EdgeServiceApplicationTests {

  private static final int REDIS_PORT = 6379;

  @Container
  static GenericContainer<?> redis =                (3)
    new GenericContainer<>(DockerImageName.parse("redis:7.0"))
      .withExposedPorts(REDIS_PORT);

  @DynamicPropertySource                            (4)
```

```
    static void redisProperties(DynamicPropertyRegistry registry) {
      registry.add("spring.redis.host",
          () -> redis.getHost());
      registry.add("spring.redis.port",
          () -> redis.getMappedPort(REDIS_PORT));
    }

    @Test
    void verifyThatSpringContextLoads() {                    ❺
    }

  }
```

❶ Loads a full Spring web application context and a web environment listening on a random port

❷ Activates automatic startup and cleanup of test containers

❸ Defines a Redis container for testing

❹ Overwrites the Redis configuration to point to the test Redis instance

❺ An empty test used to verify that the application context is loaded correctly and that a connection with Redis has been established successfully

Finally, run the integration tests as follows:

```
$ ./gradlew test --tests EdgeServiceApplicationTests
```

Should you want to disable the session management through Redis in some of your tests, you can do so by setting the spring.session.store-type property to none in a specific test class using the @TestPropertySource annotation, or in a property file if you want to make it apply to all test classes.

Polar Labs

Feel free to apply what you learned in the previous chapters and prepare the Edge Service application for deployment.

1. Add Spring Cloud Config Client to Edge Service to make it fetch configuration data from Config Service.

2. Configure the Cloud Native Buildpacks integration, containerize the application, and define the commit stage of the deployment pipeline, as you learned in chapters 3 and 6.
3. Write the Deployment and Service manifests for deploying Edge Service to a Kubernetes cluster.
4. Configure Tilt to automate the Edge Service deployment to your local Kubernetes cluster initialized with minikube.

You can refer to the Chapter09/09-end folder in the code repository accompanying the book to check the final result (**https://github.com/ThomasVitale/cloud-native-spring-in-action**). You can also deploy the backing services from the manifests available in the Chapter09/09-end/polar-deployment/kubernetes/platform/development folder with `kubectl apply -f services`.

9.5 Managing external access with Kubernetes Ingress

Spring Cloud Gateway helps you define an edge service where you can implement several patterns and cross-cutting concerns at the ingress point of a system. In the previous sections, you saw how to use it as an API gateway, implement resilience patterns like rate limiting and circuit breakers, and define distributed sessions. In chapters 11 and 12, we'll also add authentication and authorization features to Edge Service.

Edge Service represents the entry point to the Polar Bookshop system. However, when it's deployed in a Kubernetes cluster, it's only accessible from within the cluster itself. In chapter 7, we used the *port-forward* feature to expose a Kubernetes Service defined in a minikube cluster to your local computer. That's a useful strategy during development, but it's not suitable for production.

This section will cover how you can manage external access to applications running in a Kubernetes cluster using the Ingress API.

NOTE This section assumes you have gone through the tasks listed in the previous "Polar Labs" sidebar and prepared Edge Service for deployment on Kubernetes.

9.5.1 Understanding Ingress API and Ingress Controller

When it comes to exposing applications inside a Kubernetes cluster, we can use a Service object of type `ClusterIP`. That's what we've done so far to make it possible for Pods to interact with each other within the cluster. For example, that's how Catalog Service Pods can communicate with the PostgreSQL Pod.

A Service object can also be of type LoadBalancer, which relies on an external load balancer provisioned by a cloud provider to expose an application to the internet. We could define a LoadBalancer Service for Edge Service instead of the ClusterIP one. When running the system in a public cloud, the vendor would provision a load balancer, assign a public IP address, and all the traffic coming from that load balancer would be directed to the Edge Service Pods. It's a flexible approach that lets you expose a service directly to the internet, and it works with different types of traffic.

The LoadBalancer Service approach involves assigning a different IP address to each service we decide to expose to the internet. Since services are directly exposed, we don't have the chance to apply any further network configuration, such as TLS termination. We could configure HTTPS in Edge Service, route all traffic directed to the cluster through the gateway (even platform services that don't belong to Polar Bookshop), and apply further network configuration there. The Spring ecosystem provides everything we need to address those concerns, and it's probably what we would do in many scenarios. However, since we want to run our system on Kubernetes, we can manage those infrastructural concerns at the platform level and keep our applications simpler and more maintainable. That's where the Ingress API comes in handy.

An *Ingress* is an object that "manages external access to the services in a cluster, typically HTTP. Ingress may provide load balancing, SSL termination and name-based virtual hosting" (**https://kubernetes.io/docs**). An Ingress object acts as an entry point into a Kubernetes cluster and is capable of routing traffic from a single external IP address to multiple services running inside the cluster. We can use an Ingress object to perform load balancing, accept external traffic directed to a specific URL, and manage the TLS termination to expose the application services via HTTPS.

Ingress objects don't accomplish anything by themselves. We use an Ingress object to declare the *desired state* in terms of routing and TLS termination. The actual component that enforces those rules and routes traffic from outside the cluster to the applications inside is the *ingress controller*. Since multiple implementations are available, there's no default ingress controller included in the core Kubernetes distribution—it's up to you to install one. Ingress controllers are applications that are usually built using reverse proxies like NGINX, HAProxy, or Envoy. Some examples are Ambassador Emissary, Contour, and Ingress NGINX.

In production, the cloud platform or dedicated tools would be used to configure an ingress controller. In our local environment, we'll need some additional configuration to make the routing work. For the Polar Bookshop example, we'll use Ingress NGINX (**https://github.com/kubernetes/ingress-nginx**) in both environments.

NOTE There are two popular ingress controllers based on NGINX. The Ingress NGINX project (**https://github.com/kubernetes/ingress-nginx**) is developed, supported, and maintained in the Kubernetes project itself. It's open source, and it's what we'll use in this book. The NGINX Controller (**www.nginx.com/products/nginx-controller**) is a product developed and maintained by the F5 NGINX company, and it comes with free and commercial options.

Let's see how we can use Ingress NGINX on our local Kubernetes cluster. An ingress controller is a workload just like any other application running on Kubernetes, and it can be deployed in different ways. The simplest option would be using kubectl to apply its deployment manifests to the cluster. Since we use minikube to manage a local Kubernetes cluster, we can rely on a built-in add-on to enable the Ingress functionality based on Ingress NGINX.

First, let's start the `polar` local cluster we introduced in chapter 7. Since we configured minikube to run on Docker, make sure your Docker Engine is up and running:

```
$ minikube start --cpus 2 --memory 4g --driver docker --profile polar
```

Next we can enable the `ingress` add-on, which will make sure that Ingress NGINX is deployed to our local cluster:

```
$ minikube addons enable ingress --profile polar
```

In the end, you can get information about the different components deployed with Ingress NGINX as follows:

```
$ kubectl get all -n ingress-nginx
```

The preceding command contains an argument we haven't encountered yet: `-n ingress-nginx`. It means that we want to fetch all objects created in the `ingress-nginx` *namespace*.

A *namespace* is "an abstraction used by Kubernetes to support isolation of groups of resources within a single cluster. Namespaces are used to organize objects in a cluster and provide a way to divide cluster resources" (**https://kubernetes.io/docs/reference/glossary**).

We use namespaces to keep our clusters organized and define network policies to keep certain resources isolated for security reasons. So far, we've been working with the default namespace, and we'll keep doing that for all our Polar Bookshop applications. However, when it comes to platform services such as Ingress NGINX, we'll rely on dedicated namespaces to keep those resources isolated.

Now that Ingress NGINX is installed, let's go ahead and deploy the backing services used by our Polar Bookshop applications. Check the source code repository accompanying this book (Chapter09/09-end) and copy the content of the polar-deployment/kubernetes/platform/development folder into the same path in your polar-deployment repository, overwriting any existing file we used in previous chapters. The folder contains basic Kubernetes manifests to run PostgreSQL and Redis.

Open a Terminal window, navigate to the kubernetes/platform/development folder located in your polar-deployment repository, and run the following command to deploy PostgreSQL and Redis in your local cluster:

```
$ kubectl apply -f services
```

You can verify the results with the following command:

```
$ kubectl get deployment
NAME           READY   UP-TO-DATE   AVAILABLE   AGE
polar-postgres   1/1     1            1           73s
polar-redis      1/1     1            1           73s
```

TIP For your convenience, I prepared a script that performs all the previous operations with a single command. You can run it to create a local Kubernetes cluster with minikube, enable the Ingress NGINX add-on, and deploy the backing services used by Polar Bookshop. You'll find the create-cluster.sh and destroy-cluster.sh files in the kubernetes/platform/development folder that you have just copied over to your polar-deployment repository. On macOS and Linux, you might need to make the scripts executable via the chmod +x create-cluster.sh command.

Let's conclude this section by packaging Edge Service as a container image and loading the artifact to the local Kubernetes cluster. Open a Terminal window, navigate to the Edge Service root folder (edge-service), and run the following commands:

```
$ ./gradlew bootBuildImage
$ minikube image load edge-service --profile polar
```

In the next section, you'll define an Ingress object and configure it to manage external access to the Polar Bookshop system running in a Kubernetes cluster.

5.2 Working with Ingress objects

Edge Service takes care of application routing, but it should not be concerned with the underlying infrastructure and network configuration. Using an Ingress resource, we can decouple the two responsibilities. Developers would maintain Edge Service, while the platform team would manage the ingress controller and the network configuration (perhaps relying on a service mesh like Linkerd or Istio). Figure 9.6 shows the deployment architecture of Polar Bookshop after introducing an Ingress.

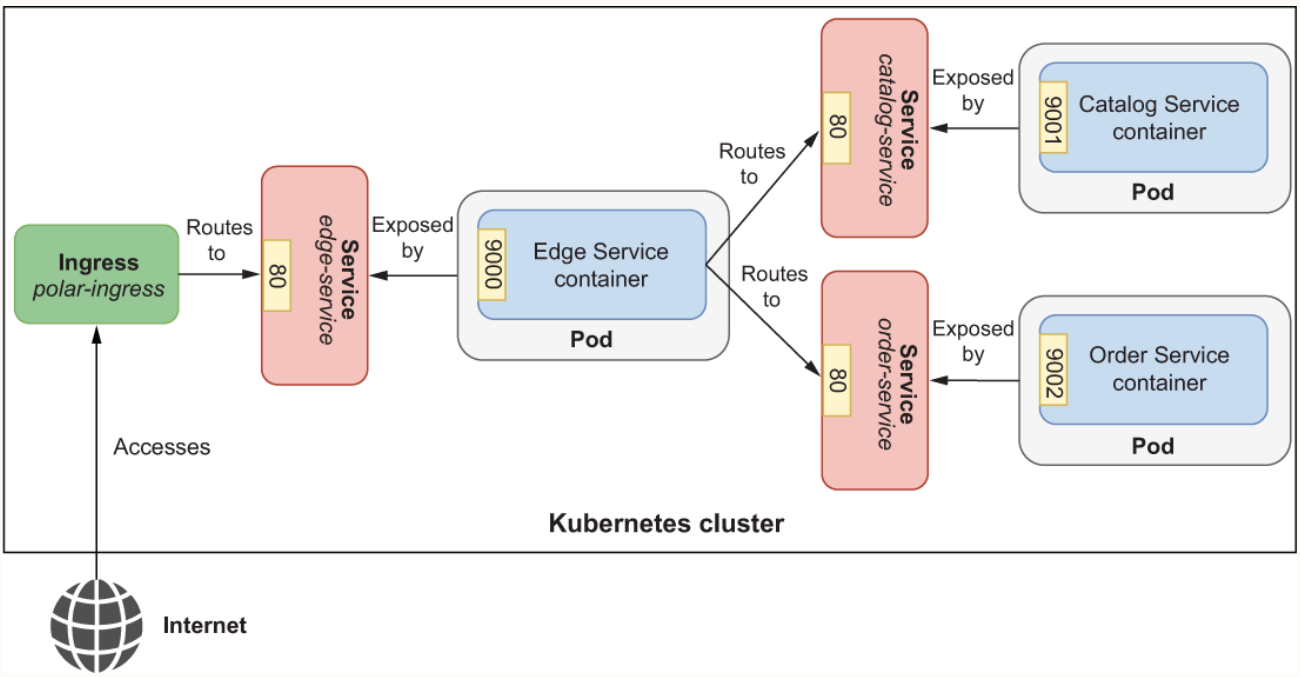


Figure 9.6 The deployment architecture of the Polar Bookshop system after introducing an Ingress to manage external access to the cluster

Let's define an Ingress to route all HTTP traffic coming from outside the cluster to Edge Service. It's common to define Ingress routes and configurations based on the DNS name used to send the HTTP request. Since we are working locally, and assuming we don't have a DNS name, we can call the external IP address provisioned for the Ingress to be accessible from outside the cluster. On Linux, you can use the IP address assigned to the minikube cluster. You can retrieve that value by running the following command:

```
$ minikube ip --profile polar
192.168.49.2
```

On macOS and Windows, the `ingress` add-on doesn't yet support using the minikube cluster's IP address when running on Docker. Instead, we need to use the `minikube tunnel --profile polar` command to expose the cluster to the local environment, and then use the `127.0.0.1` IP address to call the cluster. This is similar to the `kubectl port-forward` command, but it applies to the whole cluster instead of a specific service.

After identifying the IP address to use, let's define the Ingress object for Polar Bookshop. In the Edge Service project, create a new ingress.yml file in the k8s folder.

Listing 9.20 Exposing Edge Service outside the cluster via an Ingress

```
apiVersion: networking.k8s.io/v1        ❶
kind: Ingress                           ❷
metadata:
  name: polar-ingress                   ❸
spec:
  ingressClassName: nginx               ❹
  rules:
    - http:                             ❺
        paths:
          - path: /                     ❻
            pathType: Prefix
            backend:
              service:
                name: edge-service      ❼
                port:
                  number: 80            ❽
```

❶ The API version for Ingress objects

❷ The type of object to create

❸ The name of the Ingress

❹ Configures the ingress controller responsible for managing this object

❺ Ingress rules for HTTP traffic

❻ A default rule for all requests

❼ The name of the Service object where traffic should be forwarded

❽ The port number for the Service where traffic should be forwarded

At this point we are ready to deploy Edge Service and the Ingress to the local Kubernetes cluster. Open a Terminal window, navigate to the Edge Service root folder (edge-service), and run the following command:

```
$ kubectl apply -f k8s
```

Let's verify that the Ingress object has been created correctly with the following command:

```
$ kubectl get ingress

NAME           CLASS   HOSTS   PORTS   AGE
polar-ingress  nginx   *       80      21s
```

It's time to test that Edge Service is correctly available through the Ingress. If you're on Linux, you don't need any further preparation steps. If you're on macOS or Windows, open a new Terminal window and run the following command to expose your minikube cluster to your localhost. The command will continue running for the tunnel to be accessible, so make sure you keep the Terminal window open. The first time you run this command, you might be asked to input your machine's password to authorize the tunneling to the cluster:

```
$ minikube tunnel --profile polar
```

Finally, open a new Terminal window and run the following command to test the application (on Linux, use the minikube's IP address instead of 127.0.0.1):

```
$ http 127.0.0.1/books
```

Since Catalog Service is not running, Edge Service will execute the fallback behavior we configured earlier and return a 200 OK response with an empty body. That's what we

expected, and it proves that the Ingress configuration works.

When you are done trying out the deployment, you can stop and delete the local Kubernetes cluster with the following commands:

```
$ minikube stop --profile polar
$ minikube delete --profile polar
```

TIP For your convenience, you can also use the destroy-cluster.sh script (available in the kubernetes/platform/development folder of your polar-deployment repository) that you copied earlier from the book's source code. On macOS and Linux, you might need to make the script executable via the chmod +x destroy-cluster.sh command.

Good job! We're now ready to make Edge Service even better by adding authentication and authorization. Before configuring security, though, we still need to complete the Polar Bookshop business logic for dispatching orders. In the next chapter, you'll do that while learning event-driven architectures, Spring Cloud Function, and Spring Cloud Stream with RabbitMQ.

Summary

- An API gateway provides several benefits in a distributed architecture, including decoupling the internal services from the external API and offering a central, convenient place for handling cross-cutting concerns like security, monitoring, and resilience.
- Spring Cloud Gateway is based on the Spring reactive stack. It provides an API gateway implementation, and it integrates with the other Spring projects to add cross-cutting concerns to the application, including Spring Security, Spring Cloud Circuit Breaker, and Spring Session.
- Routes are the core of Spring Cloud Gateway. They are identified by a unique ID, a collection of predicates determining whether to follow the route, a URI for forwarding the request if the predicates allow, and a collection of filters that are applied before or after forwarding the request downstream.
- The Retry filter is for configuring retry attempts for specific routes.
- The RequestRateLimiter filter, integrated with Spring Data Redis Reactive, limits the number of requests that can be accepted within a specific time window.
- The CircuitBreaker filter, based on Spring Cloud Circuit Breaker and Resilience4J, defines circuit breakers, time limiters, and fallbacks to specific routes.

- Cloud native applications should be stateless. Data services should be used for storing the state. For example, PostgreSQL is used for persistence storage and Redis for cache and session data.
- A Kubernetes Ingress resource allows you to manage external access to applications running inside the Kubernetes cluster.
- The routing rules are enforced by an ingress controller, which is an application that also runs in the cluster.

Previous chapter

< 8 Reactive Spring: Resilience and scalability

Next chapter

10 Event-driven applications and functions >